Table of Contents

Especificação SDD — TaskAnalyzer

Versão 1.1 · Contrato executável do módulo TaskAnalyzer Bootcamp III — projeto cumulativo (Fase 1: especificação · Fase 2: implementação · Fase 3: DevOps)

> No paradigma Spec-Driven Development, este documento **é o contrato**. O código em `src/` deve aderir estritamente ao que está aqui definido. Qualquer comportamento não especificado neste documento é, por definição, comportamento proibido.

Controle do documento

Item	Conteúdo
Versão	1.1
Status	Vigente — base da implementação da Fase 2
Autor	Guilherme Borges Furtado Neves
Data	23/10/2026
Substitui	Versão 1.0, entregue na Fase 1

Histórico de versões

Versão	Data	Alteração
1.0	Fase 1	Contrato inicial: `quantidade_tarefas`, `tempo_medio_conclusao_min`, hierarquia de três exceções.
1.1	Fase 2	Alinhamento ao contrato nomeado no enunciado da Fase 2. Ver changelog abaixo.

Changelog 1.0 → 1.1

O enunciado da Fase 2 nomeia explicitamente identificadores e comportamentos que divergem da versão 1.0. Como a especificação é o contrato e o contrato mudou, a especificação foi atualizada **antes** de qualquer linha de código ser escrita. Este é o ciclo do SDD funcionando: a mudança entra pela spec, não pelo código.

#	Mudança	v1.0	v1.1	Motivo
C-01	Nome da função pública	não nomeada	`analyze_tasks`	Nomeada no item 3.2 do enunciado da Fase 2.
C-02	Hierarquia de exceções	`TaskAnalyzerError`, `InvalidTaskDataError`, `NoCompletedTasksError`	exceção única `TaskValidationError`	Nomeada no item 3.2 do enunciado da Fase 2.
C-03	Unidade de tempo	minutos (`tempo_medio_conclusao_min`)	horas (`tempo_medio_conclusao_horas`)	Nomeada no item 3.3 do enunciado da Fase 2.
C-04	Métricas de contagem	`quantidade_tarefas`	`total_tarefas`, `total_concluidas`, `total_pendentes`	Nomeadas no item 3.3 do enunciado da Fase 2.
C-05	Conjunto sem tarefas concluídas	disparava `NoCompletedTasksError`	devolve `0.0` nas médias, sem exceção	O item 3.3 exige explicitamente retorno `0.0` e ausência de exceção neste caso de borda.
C-06	Marco inicial do tempo de conclusão	`data_conclusao − data_inicio`	`data_conclusao − data_criacao`	Ver ADR-06.
C-07	Tarefa com datas inconsistentes	descartada com log `WARNING`	dispara `TaskValidationError`	O item 3.3 define `data_conclusao < data_criacao` como disparo de exceção. Com C-05 cobrindo o caso de

#	Mudança	v1.0	v1.1	Motivo
				borda, a política de dois níveis da v1.0 perdeu a razão de existir.

Consequência de C-05 e C-07 juntos. A v1.0 resolvia uma contradição do enunciado da Fase 1 com uma política de dois níveis: descartar registro ruim e disparar exceção só quando o conjunto esvaziasse. O enunciado da Fase 2 desfaz a contradição ao separar os dois casos de forma explícita — dado temporalmente inválido é exceção, ausência de dado é zero. A v1.1 adota essa separação, que é mais simples e mais testável.

---

1. Contrato Executável de Interface

1.1 Entradas

A função recebe uma **lista de tarefas**. Cada tarefa é um dicionário com os campos abaixo.

Campo	Tipo	Obrigatório	Descrição e restrições
`id_tarefa`	`int`	Sim	Identificador único. Maior que zero. Não pode se repetir na mesma lista. Valores booleanos são rejeitados.
`data_criacao`	`datetime`	Sim	Data e hora de criação, em UTC.
`data_inicio`	`datetime`	Não	Data e hora de início da execução, em UTC. `None` quando não iniciada.
`data_conclusao`	`datetime`	Não	Data e hora de conclusão, em UTC. `None` quando não concluída.
`prazo`	`datetime`	Sim	Data e hora limite

Campo	Tipo	Obrigatório	Descrição e restrições
			para conclusão, em UTC.
`prioridade`	`str` (enum)	Sim	Um de: `baixa`, `media`, `alta`. Sem acento.
`status`	`str` (enum)	Sim	Um de: `concluida`, `pendente`, `cancelada`. Sem acento.

Restrições adicionais:

- A lista pode ser vazia. É entrada válida (RN-05).
- Datas sem fuso horário são interpretadas como UTC; datas com outro fuso são convertidas (RN-06).
- Qualquer violação desta tabela dispara `TaskValidationError` (seção 1.4).

1.2 Saídas

A função devolve um dicionário com exatamente estas chaves. Nenhuma métrica adicional é produzida.

Campo	Tipo	Descrição
`total_tarefas`	`int`	Total de tarefas recebidas no recorte, independentemente do status.
`total_concluidas`	`int`	Tarefas com `status = concluida` e `data_conclusao` informada.
`total_pendentes`	`int`	Tarefas com `status = pendente`.
`tempo_medio_conclusao_horas`	`float`	Tempo médio entre criação e conclusão, em horas.
`taxa_atraso_percentual`	`float`	Percentual de concluídas entregues após o prazo.
`indicadores_por_prioridade`	`dict`	As cinco métricas acima, recalculadas para cada prioridade.

Estrutura de retorno:

```
{
  "total_tarefas": int,
  "total_concluidas": int,
  "total_pendentes": int,
  "tempo_medio_conclusao_horas": float,
  "taxa_atraso_percentual": float,
  "indicadores_por_prioridade": {
    "baixa": { "total_tarefas": int, "total_concluidas": int,
"total_pendentes": int,
               "tempo_medio_conclusao_horas": float,
"taxa_atraso_percentual": float },
    "media": { ... },
    "alta":  { ... }
  }
}
```

1.3 Assinatura pública

```
def analyze_tasks(tarefas: list[dict[str, Any]]) -> dict[str,
Any]: ...

class TaskValidationError(Exception): ...
```

Nenhum outro símbolo é exportado. `__all__` declara exatamente estes dois.

1.4 Contrato de erros

`TaskValidationError` é a única exceção lançada para o chamador. Nenhuma exceção genérica (`Exception`, `ValueError` cru) atravessa a fronteira pública.

Causa	Mensagem (padrão)
Entrada não é lista	`Entrada inválida — esperado list, recebido <tipo>`
Registro não é dicionário	`Registro inválido — esperado dict, recebido <tipo>`
`id_tarefa` ausente	`Tarefa sem identificador: id_tarefa inválido — campo obrigatório ausente`
`id_tarefa` não inteiro, menor ou igual a zero, ou duplicado	`Tarefa <id>: id_tarefa inválido — <motivo>`
Campo obrigatório ausente ou de tipo incorreto	`Tarefa <id>: <campo> inválido — <motivo>`
Enum fora do domínio	`Tarefa <id>: <campo> inválido — valor '<v>' fora do domínio [...]`
`data_conclusao` anterior a `data_criacao`	`Tarefa <id>: data_conclusao inválida — conclusão`

Causa	Mensagem (padrão)
	`anterior à criação`
`data_inicio` anterior a `data_criacao`	`Tarefa <id>: data_inicio inválida — início anterior à criação`
`data_conclusao` anterior a `data_inicio`	`Tarefa <id>: data_conclusao inválida — conclusão anterior ao início`

A validação é **eager**: o primeiro registro inválido interrompe a análise. Um lote inválido não produz resultado parcial.

1.5 Regras de negócio

ID	Regra
RN-01	Só entram no cálculo de tempo e atraso as tarefas com `status = concluida` **e** `data_conclusao` informada.
RN-02	Tempo de conclusão = `data_conclusao − data_criacao`, expresso em **horas**.
RN-03	Tarefa em atraso quando `data_conclusao > prazo`. Conclusão exatamente no prazo **não** é atraso.
RN-04	Datas fora de ordem cronológica (criação → início → conclusão) disparam `TaskValidationError`.
RN-05	Sem tarefas concluídas no recorte, `tempo_medio_conclusao_horas` e `taxa_atraso_percentual` valem `0.0`. Nenhuma exceção é lançada, e nenhuma divisão por zero ocorre.
RN-06	Fuso horário sempre UTC. Datas naive são assumidas como UTC; datas com outro fuso são convertidas.
RN-07	Arredondamento de duas casas decimais para tempos e percentuais.
RN-08	`total_tarefas` conta todas as tarefas do recorte, inclusive pendentes e canceladas.
RN-09	As métricas por prioridade usam exatamente as mesmas regras, aplicadas ao subconjunto.

ID	Regra
RN-10	As três prioridades aparecem **sempre** em `indicadores_por_prioridade`, zeradas quando vazias.
RN-11	O módulo é puro e determinístico: não persiste dados, não acessa rede, não lê o relógio do sistema e não altera a lista recebida.

1.6 Decisões de especificação (ADR)

ID	Questão	Decisão	Justificativa
ADR-01	O enunciado cita métrica "por tipo", mas não existe campo `tipo` na entrada.	Métricas agrupadas apenas por prioridade e no geral. O campo não é inventado.	O contrato de interface prevalece sobre a descrição narrativa. Inventar campo é a alucinação que o SDD previne. **Mantida da v1.0.**
ADR-03	Grafia do enum de prioridade.	`media`, sem acento.	Evita divergência de encoding entre spec, teste e código. **Mantida da v1.0.**
ADR-05	Comportamento quando a conclusão é exatamente no prazo.	Não é atraso.	Prazo é limite inclusivo. A escolha é arbitrária, mas precisa ser explícita para o teste ser escrevível. **Mantida da v1.0.**
ADR-06	O tempo de conclusão conta a partir de `data_inicio` ou de `data_criacao`?	A partir de `data_criacao`.	`data_criacao` é obrigatório e `data_inicio` não. Contar a partir da criação torna o tempo calculável para toda tarefa concluída, sem

ID	Questão	Decisão	Justificativa
			descartes silenciosos — o que é condição para o caso de borda da RN-05 devolver `0.0` de forma previsível. O enunciado da Fase 2 também ancora sua única regra temporal em `data_criacao`. `data_inicio` permanece no contrato e é usado na validação de ordem cronológica.
ADR-07	Uma exceção ou uma hierarquia de exceções?	Exceção única `TaskValidationError`.	Nomeada no enunciado da Fase 2. A granularidade perdida é recuperada na mensagem, que nomeia a tarefa, o campo e o motivo.
ADR-08	Booleano é um `int` válido em Python. Aceitar `True` como `id_tarefa`?	Rejeitar.	`isinstance(True, int)` é verdadeiro, o que deixaria passar `id_tarefa=True`. A rejeição explícita fecha a brecha.

2. Cenários de Aceite

2.1 Cenário 1 — Sucesso

> **Dado** um conjunto de tarefas válidas com diferentes prioridades, prazos e status, **quando** `analyze_tasks` for executada, **então** as métricas retornadas devem estar corretas, no geral e por prioridade.

Massa de teste de referência (todas as datas em UTC, março de 2026):

id	prioridade	status	criação	conclusão	prazo	tempo	atraso
1	alta	concluida	01 08:00	01 10:00	01 12:00	2 h	não
2	alta	concluida	01 08:00	01 14:00	01 12:00	6 h	sim
3	media	concluida	01 08:00	01 09:00	01 10:00	1 h	não
4	baixa	pendente	01 08:00	—	02 12:00	—	—
5	baixa	cancelada	01 08:00	—	02 12:00	—	—

Resultado esperado:

Recorte	total_tarefas	total_concluidas	total_pendentes	tempo_medio_conclusao_horas	taxa_atraso_percentual
Geral	5	3	1	3.0	33.33
`alta`	2	2	0	4.0	50.0
`media`	1	1	0	1.0	0.0
`baixa`	2	0	1	0.0	0.0

2.2 Cenário 2 — Exceção

> **Dado** uma entrada com datas inconsistentes ou registro malformado, **quando** `analyze_tasks` for executada, **então** deve disparar `TaskValidationError` com mensagem clara e objetiva.

Caso	Entrada	Esperado
2.a	`data_conclusao < data_criacao`	`TaskValidationError`
2.b	`data_inicio < data_criacao`	`TaskValidationError`
2.c	`data_conclusao < data_inicio`	`TaskValidationError`

Caso	Entrada	Esperado
2.d	`prioridade = "urgente"`	`TaskValidationError`
2.e	`status = "finalizada"`	`TaskValidationError`
2.f	`id_tarefa` zero, negativo, booleano ou não inteiro	`TaskValidationError`
2.g	`id_tarefa` ausente ou duplicado	`TaskValidationError`
2.h	Campo obrigatório ausente ou com tipo errado	`TaskValidationError`
2.i	Entrada não é lista, ou registro não é dicionário	`TaskValidationError`

2.3 Cenário 3 — Casos de borda

Dado uma entrada vazia ou sem tarefas concluídas, **quando** `analyze_tasks` for executada, **então** as médias e percentuais devem valer `0.0`, sem exceção.

Caso	Entrada	Esperado
3.a	Lista vazia	Todas as métricas em zero, três prioridades presentes
3.b	Apenas tarefas pendentes	`total_pendentes` correto, médias `0.0`
3.c	Apenas tarefas canceladas	`total_tarefas` correto, médias `0.0`
3.d	Conclusão exatamente no prazo	`taxa_atraso_percentual = 0.0`
3.e	Conclusão um segundo após o prazo	`taxa_atraso_percentual = 100.0`
3.f	Concluída sem `data_conclusao`	Não entra no cálculo; médias `0.0`
3.g	Datas naive ou em outro fuso	Normalizadas para UTC antes do cálculo
3.h	Lista passada duas vezes	Entrada inalterada; resultados idênticos

2.4 Matriz de rastreabilidade

Regra	Cenário	Caso de teste (pytest)
RN-01	1, 3.f	`test_total_conclui`

Regra	Cenário	Caso de teste (pytest)
		das, `test_tarefa_concluida_sem_data_conclusao_nao_entra_na_media`
RN-02	1	`test_tempo_medio_conclusao_horas_geral`
RN-03	1, 3.d, 3.e	`test_taxa_atraso_percentual_geral`, `test_conclusao_exatamente_no_prazo_nao_e_atraso`, `test_um_segundo_apos_o_prazo_e_atraso`
RN-04	2.a, 2.b, 2.c	`test_conclusao_anterior_a_criacao`, `test_inicio_anterior_a_criacao`, `test_conclusao_anterior_ao_inicio`
RN-05	3.a, 3.b, 3.c	`test_lista_vazia_retorna_zeros`, `test_apenas_pendentes_nao_dispara_excecao`, `test_apenas_canceladas_nao_dispara_excecao`
RN-06	3.g	`test_datas_sem_fuso_sao_tratadas_como_utc`, `test_datas_com_outro_fuso_sao_convertidas_para_utc`
RN-07	1	`test_arredondamento_em_duas_casas`
RN-08	1	`test_total_tarefas`
RN-09	1	`test_indicadores_por_prioridade`
RN-10	1, 3.a	`test_todas_prioridades_presentes`, `test_lista_vazia_mantem_estrutura_po`

Regra	Cenário	Caso de teste (pytest)
		`r_prioridade`
RN-11	3.h	`test_nao_modifica_a_lista_de_entrada`, `test_execucoes_repetidas_produzem_o_mesmo_resultado`
Contrato 1.2	1	`test_estrutura_da_saida`
Contrato 1.4	2.d a 2.i	`test_registro_malformado`, `test_identificador_ausente`, `test_identificador_duplicado`, `test_entrada_nao_e_lista`, `test_registro_nao_e_dicionario`, `test_mensagem_de_erro_identifica_a_tarefa`
ADR-08	2.f	`test_identificador_booleano_e_rejeitado`

2.5 Definição de pronto

- 100% dos testes do harness passando.
- Cobertura mínima de 90% das linhas de `src/task_analyzer.py`. **Atingido: 100%.**
- Todo item da matriz de rastreabilidade com teste implementado.
- Zero comportamento observável fora do que está especificado neste documento.

Princípio. Os testes derivam da especificação, nunca do código gerado. Se a implementação falha um teste, corrige-se a implementação. Adaptar o teste ao código destrói a função de controle do harness e está proibido pela regra PR-02.

---

3. Governança

As diretrizes arquiteturais, proibições e regras de interação com o assistente de IA estão em `CONTEXT_RULES.md`, fornecido como contexto persistente em toda interação de geração de código.

O plano de homologação humana, a estratégia de versionamento e o relato de uso de IA estão no README do repositório e no relatório técnico da Fase 2.